

Smart Compaction: Predicting Compaction Utility from Lakehouse Table Metadata

Jannic Alexander Cutura^{1,2} Subash Prakash³

¹European Central Bank ²DSTI School of Engineering ³IBM

The 4th International Conference on Intelligent Computing,
Communication, Networking and Services (ICCNS 2026)

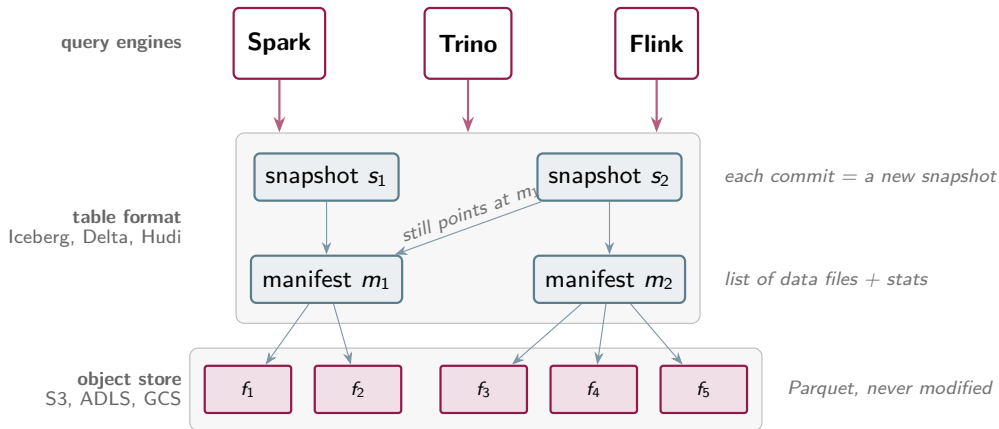
Smart Data Processing track

22–25 September 2026

Contact: jannic.cutura@dsti.institute | subash.prakash@ibm.com
Code: <https://github.com/JannicCutura/smart-compaction>

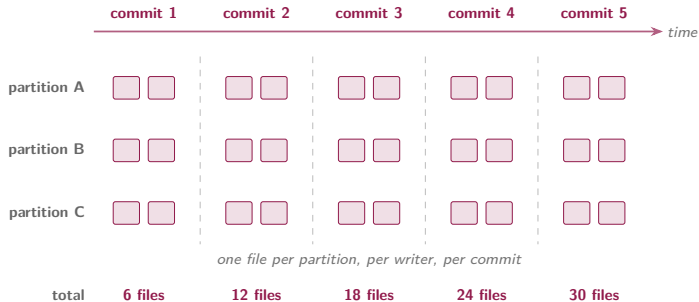
The views expressed in this presentation are solely those of the authors and do not necessarily reflect the views of the European Central Bank, the European System of Central Banks or IBM.

A lakehouse table is a set of immutable files plus a metadata tree



Engines see a table through its metadata, not its files. Every commit **appends** new files and a new snapshot and never touches existing ones — that is what buys ACID, time travel and snapshot isolation on cheap object storage.

Every commit appends files, nothing ever merges them



files = partitions × writers × commits.

Our grid tops out at $100 \times 5 \times 20 = 10,000$ files for a single table
— most of them far below the size a scan wants.

Why file count hurts



Planning

every manifest entry is read per query



Object store

LIST and GET are billed per request

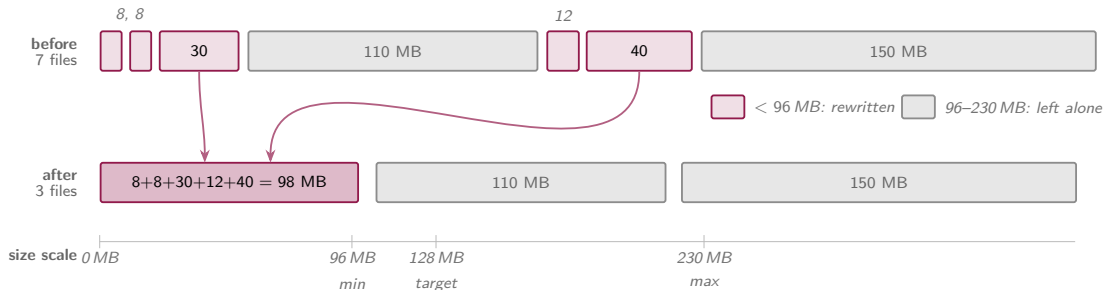


Scan

open + footer read + task, per file

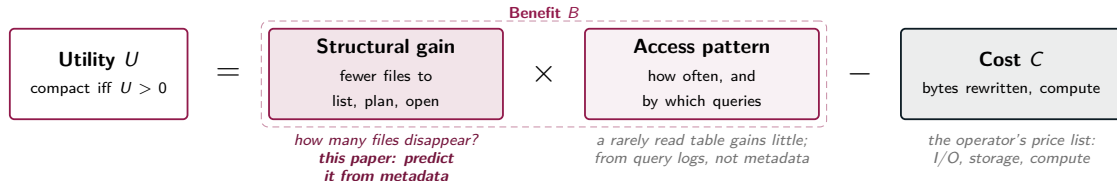
At scale: LinkedIn's AutoComp manages 35,000+ Iceberg tables; catalogs accumulate **millions** of undersized files.

Compaction rewrites many small files into a few target-sized ones



Iceberg's `rewrite_data_files` with **bin-packing**: files below the minimum are packed into target-sized outputs; files already in range are skipped. Reading and rewriting those bytes is the price — so the question is not *whether* to compact, but *when it pays*.

Compact or not? Utility is benefit minus cost



We measure the structural gain as the **file-reduction ratio**

$$r_{fc} = 1 - \frac{n_{\text{after}}}{n_{\text{before}}} \in [0, 1]$$

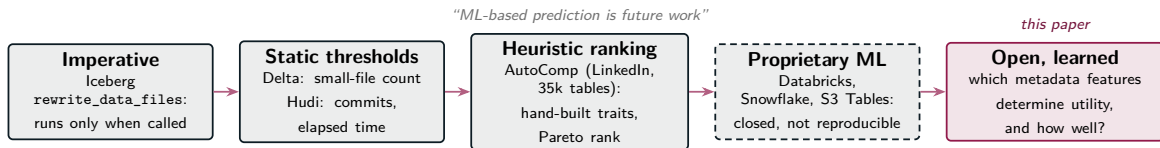
$r_{fc} \approx 1$: the table collapses to a few files. $r_{fc} \approx 0$: the rewrite is wasted I/O. On the previous slide, $r_{fc} = 1 - 3/7 = 0.57$.

Two questions for the metadata

- 1 **Whether:** will compaction rewrite anything at all?
- 2 **How much:** what r_{fc} will it achieve?

Access pattern and cost are the operator's; we supply the structural signal.

Today the decision is a rule; we make it a prediction



Every open scheduler fires on a fixed rule; no published study asks *which* metadata determines compaction utility.

Target

$$r_{fc} = 1 - \frac{n_{\text{after}}}{n_{\text{before}}} \in [0, 1]$$

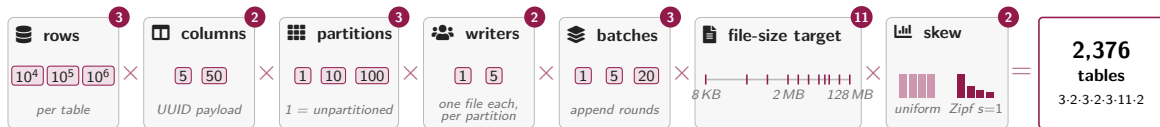
measured by running Iceberg's bin-packing
rewrite_data_files on every table

Two tasks, from metadata only

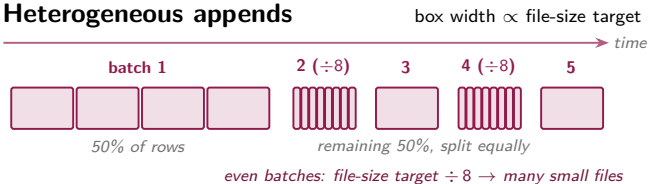
- 1 **Classification:** will bin-packing rewrite any file?
- 2 **Regression:** predict r_{fc} , for cost-aware scheduling

Inputs: manifests, snapshots, partition stats — no data file is read.

Simulation: 2,376 synthetic tables, each compacted once and labelled



Heterogeneous appends



One table, two file-size regimes: mean file size alone no longer gives the label away — the **tautology trap**.

Per table



Generate

Spark writes one Iceberg table per grid point



Extract

17 features from manifests, no data file opened



Compact

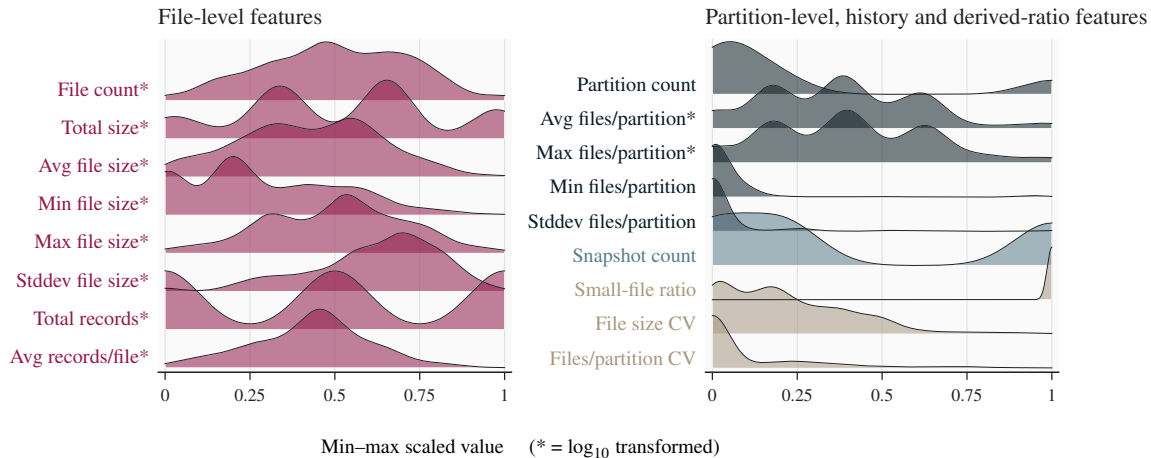
bin-pack to 128 MB; 96–230 MB left alone



Label

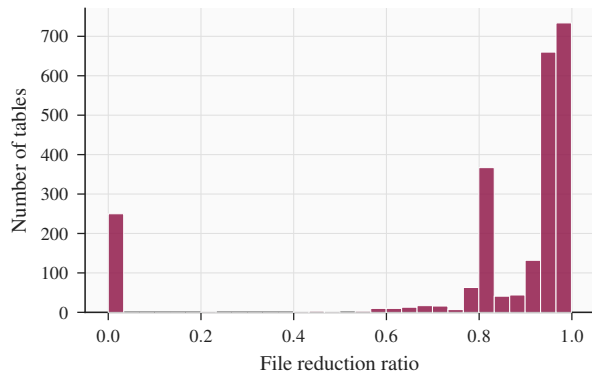
$r_{fc} = 1 - n_{\text{after}} / n_{\text{before}}$; class: rewritten > 0

17 Metadata Features



Min-max scaled to [0, 1]; * = log₁₀-transformed. Read from manifests alone — no data file is opened.

What the Simulation Produced



Distribution of r_{fc} across all 2,376 tables.

- **2,376** tables generated
- **2,121** (89.3%) had at least one file rewritten
- r_{fc} : mean 0.818, median 0.950, $\sigma = 0.294$

Bimodal, not continuous

Either compaction does essentially nothing, or it does a lot. Only **11 of 2,376** tables land in $(0, 0.5]$.

So the binary label and the continuous target nearly coincide — which is the first hint that *whether* is the easy question and *how much* is the interesting one.

Classification: Binary Decision

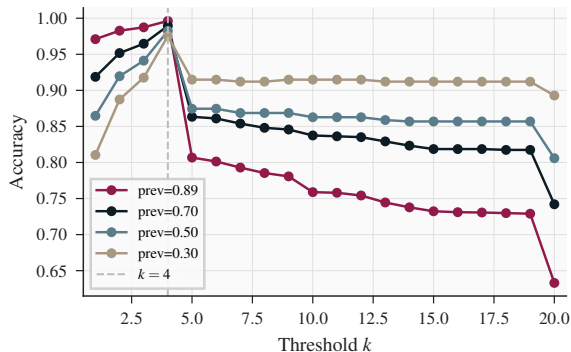
Classification (held-out test, $n = 476$)

Model	Acc.	F1
XGBoost	1.000	1.000
Logistic Reg.	0.815	0.885
Threshold ($k=4$)	1.000	1.000

The whole classifier

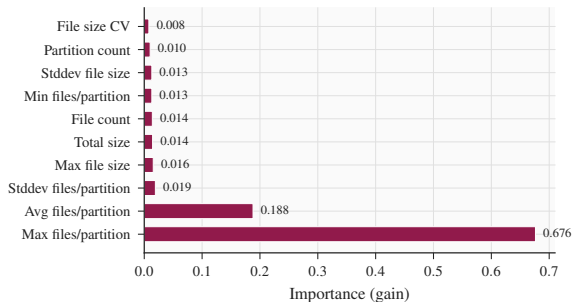
```
max_files_per_partition > 4
```

A single threshold matches XGBoost exactly. No learned model required.

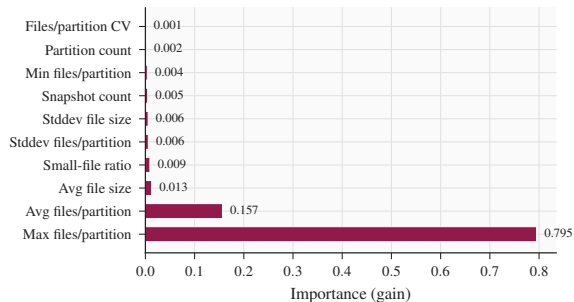


Accuracy peaks at $k = 4$ and collapses at $k = 5$ — a hard boundary, not a fitted one. Holds at every prevalence level.

Where the Signal Lives



Classifier — **86.4%** of gain



Regressor — **95.2%** of gain

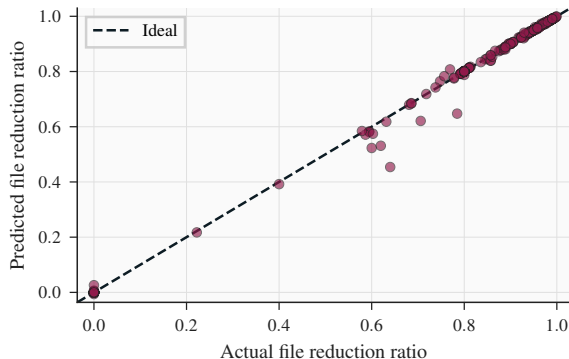
Two partition-level counts dominate both tasks. Every other feature contributes ≤ 0.019 gain.

Regression: Continuous Prediction

Regression (held-out test)

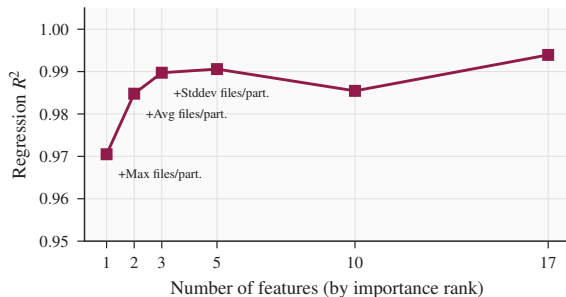
Model	RMSE	R^2
XGBoost	0.013	0.998
OLS	0.236	0.361

- OLS cannot capture the nonlinear saturation curve
- Two features account for **95.2%** of regressor gain
- Spearman $\rho = 0.965$: monotonic but not linear



Predicted vs. actual r_{fc} on the held-out test set.

Is the Result Fragile?



One feature alone reaches $R^2 = 0.970$; two reach 0.985. Past five, returns diminish.

Four robustness checks:

- **Ablation:** the other 15 features add only $\Delta R^2 = 0.023$ (95% CI [0.004, 0.044])
- **Leave-one-out:** every single removal still keeps $R^2 \geq 0.988$
- **30 bootstrap splits:** $R^2 = 0.999$, CI [0.996, 1.000]
- **Prevalence 89% $\rightarrow$ 30%:** regression $R^2 > 0.995$ throughout

Holds across model families too — random forest 0.998, decision tree 0.997. Any tree-based model suffices; only the linear baseline fails.

TPC-H Validation (No Retraining)

96 Iceberg tables from 6 TPC-H schemas (lineitem, orders, customer, part, partsupp, supplier)

- Realistic data types (strings, dates, decimals)
- 7–16 columns per schema
- Scale factor ~ 10 (468.8 M rows, 15 GB)

Cross-schema results

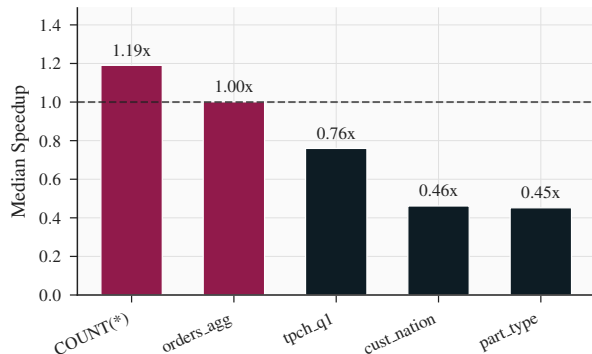
Task	Acc./RMSE	F1/ R^2
Clf (XGBoost)	0.979	0.987
Clf (Threshold $k=4$)	0.979	0.987
Clf ($sfr > 0$)	0.833	0.908
Clf (Majority class)	0.823	0.903
Reg (XGBoost)	0.058	0.976

Key findings:

- Transfers with **no retraining** — no TPC-H table was seen during training
- XGBoost and the $k=4$ threshold tie at 97.9%, missing the same two borderline tables (3 files at ~ 50 MB)
- The learned model earns its keep on the *continuous* target ($R^2 = 0.976$), not the binary decision

Weakest schemas are `customer` and `partsupp` ($R^2 \approx 0.87$), which produce tables sitting closest to the compaction threshold.

Does Compaction Actually Make Queries Faster?



52 TPC-H tables, Iceberg time-travel (VERSION AS OF) to query both snapshots. Median of 3 timed runs.

Not universally.

- COUNT(*): **1.19**× faster — metadata overhead dominates
- Filtered aggregation: neutral to 0.76×
- Full-scan aggregation: **0.45**× — 2× *slower*

Why

Spark assigns tasks per file. Compacting 300 files into 1 drops parallelism from 300 tasks to 1, offsetting the metadata saving for queries that must read everything.

Overall median across 104 measurements: **0.97**×.

Key Insights

Tautology Trap

Our first grid produced only small files (max ~ 0.4 MB vs. the 96 MB threshold), so *every* table compacted completely: $r_{fc} = 1 - 1/N$. XGBoost scored $R^2 = 0.99$ by recovering that identity ($\rho = 0.996$). Any benchmark using only small synthetic files will produce misleadingly perfect metrics.

Feature Dominance

Partition-level file counts (`max_files_per_partition`, `avg_files_per_partition`) dominate both tasks.

- Binary decision: fully solvable by a single threshold
- Continuous prediction: requires nonlinear modelling

Practical Implication

AutoComp's compaction traits already include these features. They are sufficient for the binary decision — but **cost-aware scheduling** needs the continuous prediction. A workable rule: compact when predicted $r_{fc} > 0.3$, which drops the lowest-utility jobs while keeping 99.8% of the beneficial ones.

Threats to Validity

Setup:

- Synthetic UUID payloads compress differently from real columnar data
- Partition structure generated, not domain-derived; query patterns not modelled
- Spark-local execution $\neq$ distributed clusters with S3
- 89.3% prevalence may inflate accuracy — mitigated by majority-class baseline, downsampling, bootstrap CIs

Scope — what we did *not* test:

- **Delete files / MoR:** we use Copy-on-Write only. Merging delete files is a different dynamic our features do not capture
- **Sort and z-order:** bin-packing only. Their utility depends on query workload, which metadata cannot see
- **Temporal evolution:** static snapshots; a table that is fine now may need compaction after the next write
- **One compaction target:** $k=4$ is tied to the 128 MB/96 MB configuration

Conclusion

- 1 **Binary compaction decision** is trivially solvable by a single partition-level threshold
- 2 **Continuous reduction ratio** requires nonlinear modelling (XGBoost $R^2 = 0.998$ vs. OLS $R^2 = 0.361$)
- 3 **Transfers across schemas** without retraining: 97.9% on TPC-H (threshold and XGBoost alike), regression $R^2 = 0.976$
- 4 Compaction utility is determined by **structural metadata**, not data content
- 5 Downstream benefit is **workload-dependent**: metadata-heavy queries speed up, full-scan aggregations slow down

Future work:

- Extend to Delta Lake and Hudi
- Add query-workload features and cloud-cost metrics
- Multi-table scheduling via contextual bandits
- Sort and z-order compaction strategies
- Validation on production Iceberg catalogs

All code and data publicly available.

Thank You

Questions?

Simulation framework, models and code:

<https://github.com/JannicCutura/smart-compaction>

Backup: Per-Schema TPC-H Breakdown

Schema	n	Clf Acc.	Reg R^2
lineitem	16	1.000	0.997
orders	16	1.000	1.000
part	16	1.000	1.000
supplier	16	1.000	1.000
customer	16	0.938	0.874
partsupp	16	0.938	0.867

`customer` and `partsupp` each contribute one misclassified table. Both sit near the 96 MB compaction threshold, where a small file-size shift flips the outcome.

Backup: Query Latency by Type

Query	n	Median	Mean	ρ	p
count_star	52	1.19×	1.31×	0.50	<0.001
orders_agg	12	1.00×	1.63×	0.89	<0.001
tpch_q1	13	0.76×	0.82×	0.77	0.002
cust_nation	15	0.46×	0.56×	0.45	0.095
part_type	12	0.45×	0.47×	0.38	0.226

52 TPC-H tables, median of 3 timed runs after 1 warmup. Speedup $> 1\times$ means faster after compaction. ρ is the Spearman correlation with the file-reduction ratio.